



DAA

Thứ ngày . . .

VD1:

a) Xét hàm $f(n) = 4n + 5$. Ta cần tìm $c > 0$ và n_0 sao cho:

~~$$4 \leq 4n + 5 \leq cn, \forall n \geq n_0$$~~

~~$$4n + 5 \leq cn \Rightarrow 5 \leq n(c - 4)$$~~

Ta có: $f(n) = 4n + 5 \leq 8n$ với $n \geq 5$. $\Rightarrow \exists c = 5, n_0 = 5, g(n) = 5$ sao cho $f(n) \leq c \cdot g(n) \Rightarrow f(n) \leq 5n$ với $\forall n \geq 5 \Rightarrow$ Tìm cận trên của $f(n) = 4n + 5 = O(g(n)) = O(n)$.b) $f(n) = n^2 + 2 = O(n^2)$.

$$n^2 + 2 \leq 2n^2 \text{ với } n \geq \sqrt{2}.$$

c) $f(n) = n^3 + 100n^2 + 50$

$$n^3 + 100n^2 + 50 \leq 2n^4 \text{ với } n \geq 11$$

 $\Rightarrow$ Tìm cận trên của $f(n) = n^3 + 100n^2 + 50 = O(g(n)) = O(n^4)$ d) $f(n) = 508$.

$$508 \leq 508, \Rightarrow c = 508, g(n) = 1 \forall n, n_0$$

 $\Rightarrow$ TCT của $f(n) = 508 = O(1)$.VD2: $O(1)$.VD3: $O(n)$.VD4: $O(1)$.VD5: $O(n)$. $O(\log n)$.VD6: $O(n^2)$.

VD7:

int $i, j, k, \text{count} = 0$;for ($i = n/2, i \leq n; i++$) $O(n)$ for ($j = 1, j + n/2 \leq n, j++$) $O(n)$ for ($k = 1; k \leq n, k = k * 2$) $O(\log n)$.

count++

 $\rightarrow O(n^2 \log n)$

HONGHA

VD8:

mt i, j, k, count = 0		
for (i = n/2, i ≤ n; i++)		$O(n)$.
for (j = 1, j ≤ n; j = 2*j)		$O(\log n)$.
for (k = 1, k ≤ n, k = k*2)		$O(\log n)$.
count++		$\rightarrow O(n \log^2 n)$.

VD9:

a) $T(n) = n \log n + 3n + 2$!
 - Thời gian chi phối: $n \log n$.
 $\rightarrow T(n) = O(n \log n)$.

b) $T(n) = n \log(n!) + 5n^2 + 6$.
 - Sử dụng xấp xỉ Stirling: $\log(n!) \approx n \log n - n$, ta có:
 $n \log(n!) \approx n(n \log n - n) = n^2 \log n - n^2$
 - Thời gian chi phối: $n^2 \log n$.
 $\rightarrow$ kết luận: $T(n) = n^2 \log n$.

c) $T(n) = 100n + 10n^2$.
 - Thời gian chi phối: n^2 .
 $\rightarrow T(n) = O(n^2)$.

d) $T(n) = 50n \log n + n^3 + 100n$.
 - Thời gian chi phối: n^3 .
 $\rightarrow T(n) = O(n^3)$.

e) $T(n) = 10n \log n + n(\log n)^2$.
 Thời gian chi phối: $n(\log n)^2$.
 $\rightarrow T(n) = O(n(\log n)^2)$.

VD10:

a) mt n = arr.length, total = 0		$O(1)$.
for (int j = 0, j < n, j++)		$O(n)$.
total += arr[j]		$O(1)$.
return total		$\rightarrow O(n)$.



Thứ ngày .

b) int n = arr.length, total = 0	$O(1)$
for (int j = 0, j < n; j++)	$O(n/2)$
total += arr[j]	$O(1)$
return total	$\rightarrow O(n)$

c) int n = arr.length, total = 0	$O(1)$
for (int j = 0, j < n, j++)	$O(n)$
for (int k = 0, k < j, k++)	$O(n)$
total += arr[j]	$O(1)$
return total	$\rightarrow O(n^2)$

d) int n = arr.length for (int j = 0, j < n, j++)	$O(n)$
prefix += arr[j]	$O(1)$
total += prefix	$O(1)$
return total	$\rightarrow O(n)$

e) for (int i = 0, i < n, i++)	$O(n)$
for (int j = 0, j < n, j++)	$O(n)$
for (int k = 0, k < j, k++)	$O(n)$
	$\rightarrow O(n^3)$

BTVN. Ex 1:

3.

- Worst case: Khi khóa nằm ở cuối hoặc ko nằm trong list thì thuật toán phải so sánh hết phần tử trong dãy do đó phức tạp $O(n)$
- Average case: Khóa nằm bất cứ đâu trong list do đó phức tạp $O(n)$.

- Best case: Khóa nằm đầu list do đó phức tạp $O(1)$.

6

- Đôi với bất kỳ thuật toán sắp xếp nào, hoặc khi sắp xếp ta phải xem list đã dc sắp xếp hay chưa. Nếu đã được sắp xếp, chỉ cần trả ra kết quả yêu cầu.
- Còn tùy vào bài toán mà cách này có thể hiệu quả. Nếu

Kích thước đầu vào quá lớn, đôi khi việc kiểm tra còn khiến thuật toán chạy lâu hơn. Nên ta có thể sử dụng đtg trong TH này.

1.

a) Computing the average of n number.

i) n .

ii) Phép cộng 2 số.

iii) Không.

b) Computing $n/n!$

i) n .

ii) Phép nhân 2 số.

iii) Không.

c) Finding the smallest element in a list of n number.

i) n .

ii) Phép so sánh 2 số.

iii) Không.

d) Reverse display a list of n number.

i) n .

ii) Phép chia modulo.

iii) Có.

e) Reverse a list of n number.

i) n .

ii) Đổi chỗ 2 số.

iii) Không.

f) pen-and-pencil algorithm for adding 2 digit decimal Int

i) n .

ii) Phép cộng 2 chữ số và chuyển phân bù sang cột tiếp theo

iii) Không.

g.

a) $n \log(2n)$.



$$\begin{aligned}
 F(3n) - F(n) &= 3n \log 3n - n \log 2n \\
 &= 2n \log 2 + 3n \log 3 + 2n \log n \\
 &= O(n \log n) \Rightarrow \text{Giá trị tăng xấp xỉ } O(n \log n)
 \end{aligned}$$

b) n .

$$F(3n) = 3n = 3F(n).$$

- Giá trị hàm tăng 3 lần.

c) n^2 .

$$F(3n) = (3n)^2 = 9n^2 = 9F(n).$$

- Giá trị hàm tăng 9 lần.

d) n^3 .

$$F(3n) = (3n)^3 = 27n^3 = 27F(n).$$

- Giá trị hàm tăng 27 lần.

e) $n!$.

$$F(3n) = (3n)!$$

- Giá trị hàm sẽ tăng rất lớn vô khả tử khi n lớn theo giai thừaf) 2^n .

$$F(3n) = 2^{3n} = 2^{2n} \cdot 2^n = 4^n \cdot F(n)$$

Giá trị của hàm tăng 4^n lần

2. a) Phép toán cơ bản là từ 2 ma trận

$$C[i, j] = A[i, j] - B[i, j]. \quad \forall 1 \leq i, j \leq n.$$

- Số lần phép toán thực hiện

Vì mỗi ma trận có n^2 phần tử nên cần thực hiện mỗi phép toán tổng số phép tính là n^2 .- Dưới dạng hàm của ma trận bậc n : Số phép toán là $O(n^2)$.

- Dưới dạng hàm tổng của số phần tử đầu vào

Vì tổng số phần tử của 2 ma trận là $2n^2$ nên số phép toán cơ bản tỉ lệ với số phần tử trong một ma trận, tức $O(N)$ với $N = n^2$ là tổng số phần tử trong một ma trận.

b) Tính nghịch đảo của ma trận $n \times n$, kí hiệu A^{-1} có ph. khử Gauss

- Phép toán cơ bản:
- + Chia (bước chuẩn hoá tạo phép biến đổi hàng).
- + Nhân và trừ (Phép khử).
- Số lần thực hiện
- + Số phép toán là $O(n^3)$ bao gồm quá trình khử tiên ($O(n^3)$) và thế lưu ($O(n^3)$) $\Rightarrow$ Độ phức tạp $O(n^3)$.
- Dãy hàng của bảng ma trận n : $O(n^3)$.
- Dãy hàng của tổng số ptử N :
- Vì $N = n^2$, số phép toán là $O(N^{3/2})$.

Ex2.

1.

- Worst case: $\Theta(n)$ (~~$O(n)$~~)
- Best case ~~$\Theta(n)$~~ $\Omega(n)$. $\Omega(x)$.
- Average case ~~$\Theta(n)$~~ . $\Omega(x)$.

2.

a) True.

b) True.

c) False.

d) True.

5.

- $(\ln n)^3 \rightarrow O((\ln n)^3)$.
- $2 \lg(n+50)^5 \rightarrow O(\lg n)^5$.
- $\sqrt{n} \rightarrow O(n^{0,5})$.
- $0,05n^{10} + 3n^3 + 1 \rightarrow O(n^{10})$.
- $3^{2n} \rightarrow O(3^{2n})$.
- $3^{3n} \rightarrow O(3^{3n})$.
- $(n^2 + 3)! \rightarrow O(n^2)!$.

$\Rightarrow$ Thứ tự độ tăng trưởng từ thấp đến cao:

$(\ln n)^5, 2 \lg(n+50)^5, \sqrt{n}, n^{10}, 3^{2n}, 3^{3n}, (n^2+3)!$



6.

a) Theo định nghĩa $O(n^k)$, ta tìm c_1, n_0 sao choKhi n đủ lớn $p(n) \leq c_1 n^k$. $\forall n \geq n_0$.Khi n đủ lớn, bậc cao nhất $a_k n^k$ thống trị các số hạng khác $\Rightarrow$ $\exists C > 0$ sao cho: $p(n) \leq (a_k + |a_{k-1}| + \dots + |a_0|)n^k = c_1 n^k$,với $c_1 = a_k + |a_{k-1}| + \dots + |a_0| \Rightarrow p(n) \in O(n^k)$.Theo định nghĩa $\Omega(n^k)$, ta tìm c_2 và n_1 sao cho: $p(n) \geq c_2 n^k$. $\forall n \geq n_1$.Chọn $c_2 = \frac{a_k}{2}$ ta có: $p(n) = a_k n^k + \dots$ với $p(n) \geq \frac{a_k}{2} n^k$ khi n đủ lớn $\Rightarrow p(n) \in \Omega(n^k)$.Vì $p(n) \in O(n^k)$ và $p(n) \in \Omega(n^k) \Rightarrow p(n) \in \Theta(n^k)$.

Ex3:

a) Mã giả:

selection Sort (array a).

 $n = \text{length}(a)$.for $i = 0$ to $n-2$. $\text{min_index} = i$ for $j = i+1$ to $n-1$. if $a[j] < a[\text{min_index}]$. $\text{min_index} = j$. if $\text{min_index} \neq i$ swap ($a[i], a[\text{min_index}]$).

return a.

b)

Chỉ cần thực hiện với $n-1$ phần tử đầu vì phần tử cuối là phần tử có giá trị lớn nhất nên ta chỉ cần duyệt mảng đến $n-2$.

c)



- Worst case: Mảng đã sắp xếp nhưng theo thứ tự ngược lại. Khi đó độ phức tạp là $O(n^2)$ và số phép đổi đổi là n .
- Best case: Mảng đã sắp xếp và độ phức tạp là $O(n^2)$ nhưng phép đổi đổi là 0.

d) Viết chương trình.

```
selSort (array a).
```

```
  n = length(a).
```

```
  for i = 0 to n-2.
```

```
    min_index = i
```

```
    for j = i+1 to n-1.
```

```
      if a[j] < a[min_index]
```

```
        min_index = j.
```

```
    if min_index != i
```

```
      swap (a[i], a[min_index]).
```

```
  return a.
```